

Технологии параллельного программирования

MPI

MPI

- MPI (Message Passing Interface) — интерфейс передачи сообщений между процессами, выполняющими одну задачу. Он предназначен, в первую очередь, для систем с распределенной памятью (MPP) в отличие от OpenMP.
- Стандартизацией MPI занимается MPI Forum.
- Существуют реализации MPI под большинство современных платформ, операционных систем и языков.
- MPI широко применяется при решении различных задач вычислительной физики, фармацевтики, материаловедения, генетики и других областей знаний.

Существующие реализации

- Минимально в состав MPI входят: библиотека программирования (заголовочные и библиотечные файлы для языков Си, Си++ и Фортран) и загрузчик приложений.
- MPICH,
- WinMPICH,
- WMPI,
- PowerMPI
- HPMPI

Требования для использования отладчика MPI-кластера

- На компьютере разработчика должна быть установлена среда Visual Studio 2010 Professional Edition или более высокой версии (с удаленным отладчиком).
- У пользователя должны быть права администратора кластера.
- У Visual Studio должен быть доступ ко всем вычислительным узлам, на которых предполагается запускать сеансы отладки.

Необходимый доступ обеспечивают сценарии:

- Приложение разрабатывается на головном узле кластера или на выделенном узле входа.
- Используется кластер, в котором вычислительные узлы подключены к корпоративной сети (топология 2, 4 или 5), а компьютер разработчика присоединен к тому же домену или к домену, для которого настроены отношения доверия с доменом кластера.

- Для отправки приложения в кластер HPC с клиентского компьютера необходимо установить пакет **Microsoft HPC Pack 2008**.
- Для построения MPI-программ с использованием интерфейса передачи сообщений корпорации Майкрософт необходимо, чтобы на компьютере разработчика был установлен пакет **Windows HPC Server 2008 SDK**.

- MPI-программа — это набор процессов, которые могут посылать друг другу сообщения посредством различных MPI-функций.
- Каждый процесс имеет специальный идентификатор — ранг (rank).
- Ранг процесса может использоваться в различных операциях отправки MPI-сообщений, например, ранг можно указать в качестве идентификатора получателя сообщения.

- При запуске приложения все процессы помещаются в создаваемую для приложения общую область связи.
- При необходимости они могут создавать новые области связи на базе существующих. Все области связи имеют независимую друг от друга нумерацию процессов.

- Кроме того в MPI существуют специальные объекты, называемые **коммуникаторами** (communicator), описывающие группы процессов. Каждый процесс в рамках одного коммуникатора имеет уникальный ранг.
- Один и тот же процесс может относиться к разным коммуникаторам и, соответственно, может иметь разные ранги в рамках разных коммуникаторов.
- Каждая операция пересылки данных в MPI должна выполняться в рамках какого-то коммуникатора.
- По умолчанию всегда создается коммуникатор MPI_COMM_WORLD, в который входят все имеющиеся процессы. Он описывает стартовую область связи, объединяющую все процессы приложения.

Категории функций

- **Блокирующие** - останавливают (блокируют) выполнение процесса до тех пор, пока производимая ими операция не будет выполнена.
- Неблокирующие функции возвращают управление немедленно, а выполнение операции продолжается в фоновом режиме; за завершением операции надо проследить особо.
- Неплокирующие функции возвращают квитанции ("requests"), которые погашаются при завершении.
- До погашения квитанции с переменными и массивами, которые были аргументами неблокирующей функции, ничего делать нельзя.

Категории функций

- **Локальные** - не инициируют пересылок данных между ветвями. Большинство информационных функций является локальными, т.к. копии системных данных уже хранятся в каждой ветви.
- Функция передачи `MPI_Send` и функция синхронизации `MPI_Barrier` **не являются** локальными, поскольку производят пересылку.
- Следует заметить, что функция приема `MPI_Recv` (парная для `MPI_Send`) является локальной: она всего лишь пассивно ждет поступления данных, ничего не пытаясь сообщить другим ветвям.

Категории функций

- **Коллективные** - должны быть вызваны ВСЕМИ ветвями-абонентами того коммуникатора, который передается им в качестве аргумента.
- Несоблюдение для них этого правила приводит к ошибкам на стадии выполнения программы (как правило, к зависанию программы).

Принятая в MPI нотация записи

- Регистр букв важен в Си и C++.
- Все идентификаторы начинаются с префикса "MPI_". Это правило без исключений. Не рекомендуется заводить пользовательские идентификаторы, начинающиеся с этой приставки, а также с приставок "MPID_", "MPIR_" и "PMPI_", которые используются в служебных целях.
- Если идентификатор сконструирован из нескольких слов, слова в нем разделяются подчеркиками: MPI_Get_count, MPI_Comm_rank. Иногда, однако, разделитель не используется: MPI_Sendrecv, MPI_Alltoall.

- Порядок слов в составном идентификаторе выбирается по принципу "от общего к частному": сначала префикс "MPI_", потом название категории (Type, Comm, Group, Attr, Errhandler и т.д.), потом название операции (MPI_Errhandler_create, MPI_Errhandler_set, ...). Наиболее часто употребляемые функции выпадают из этой схемы: они имеют короткие и стереотипные названия, например MPI_Barrier, или MPI_Unpack.
- Имена констант (и неизменяемых пользователем переменных) записываются полностью заглавными буквами: MPI_COMM_WORLD, MPI_FLOAT. В именах функций первая за префиксом буква - заглавная, остальные маленькие: MPI_Send, MPI_Comm_size.

Обрамляющие функции

1. Инициализация библиотеки. Одна из первых инструкций в функции `main` (главной функции приложения):

```
MPI_Init( &argc, &argv );
```

- Она получает адреса аргументов, стандартно получаемых самой `main` от операционной системы и хранящих параметры командной строки. В конец командной строки программы MPI-загрузчик `mpirun` добавляет ряд информационных параметров, которые требуются `MPI_Init`.

2. Аварийное закрытие библиотеки. Вызывается, если пользовательская программа завершается по причине ошибок времени выполнения, связанных с MPI:

`MPI_Abort( описатель области связи, код ошибки MPI );`

- Вызов `MPI_Abort` из любой задачи принудительно завершает работу ВСЕХ задач, подсоединенных к заданной области связи. Если указан описатель `MPI_COMM_WORLD`, будет завершено все приложение (все его задачи) целиком, что, по-видимому, и является наиболее правильным решением. Используйте код ошибки `MPI_ERR_OTHER`, если не знаете, как охарактеризовать ошибку в классификации MPI.

3. Нормальное закрытие библиотеки:

`MPI_Finalize( ) ;`

Настоятельно рекомендуется не забывать вписывать эту инструкцию перед возвращением из программы, то есть:

- перед вызовом стандартной функции Си **exit** ;
- перед каждым оператором **return** после `MPI_Init` в функции `main` ;
- если функции `main` назначен тип `void`, и она не заканчивается оператором `return`, то `MPI_Finalize()` следует поставить в конец `main`.

- Две информационных функции: сообщают размер группы (то есть общее количество задач, подсоединенных к ее области связи) и порядковый номер вызывающей задачи:

```
int size, rank;
```

```
MPI_Comm_size( MPI_COMM_WORLD, &size );
```

```
MPI_Comm_rank( MPI_COMM_WORLD, &rank );
```

Связь "точка-точка"

- Это самый простой тип связи между задачами: одна ветвь вызывает функцию передачи данных, а другая - функцию приема. В MPI это выглядит так:

Задача 1 передает:

```
int buf[10];
```

```
MPI_Send( buf, 5, MPI_INT, 1, 0, MPI_COMM_WORLD );
```

Задача 2 принимает:

```
int buf[10];
```

```
MPI_Status status;
```

```
MPI_Recv(buf, 10, MPI_INT, 0, 0, MPI_COMM_WORLD,  
&status );
```

Аргументы функций:

- Адрес буфера, из которого в задаче 1 берутся, а в задаче 2 помещаются данные. Помните, что наборы данных у каждой задачи свои, поэтому, используя одно и то же имя массива в нескольких задачах, Вы указываете не одну и ту же область памяти, а разные, никак друг с другом не связанные.
- Размер буфера. Задается не в байтах, а в количестве ячеек. Для MPI_Send указывает, сколько ячеек требуется передать (в примере передаются 5 чисел). В MPI_Recv означает максимальную емкость приемного буфера. Если фактическая длина пришедшего сообщения меньше - последние ячейки буфера останутся нетронутыми, если больше - произойдет ошибка времени выполнения.
- Тип ячейки буфера. MPI_Send и MPI_Recv оперируют массивами однотипных данных. Для описания базовых типов Си в MPI определены константы MPI_INT, MPI_CHAR, MPI_DOUBLE и так далее, имеющие тип MPI_Datatype. Их названия образуются префиксом "MPI_" и именем соответствующего типа (int, char, double, ...), записанным заглавными буквами. Пользователь может "регистрировать" в MPI свои собственные типы данных, например, структуры, после чего MPI сможет обрабатывать их наравне с базовыми.

- Номер задачи, с которой происходит обмен данными. Все задачи внутри созданной MPI группы автоматически нумеруются от 0 до (размер группы-1).
- Идентификатор сообщения. Это целое число от 0 до 32767, которое пользователь выбирает сам. Оно служит той же цели, что и, например, расширение файла.
- задача-приемник: по идентификатору определяет смысл принятой информации ;
- сообщения, пришедшие в неизвестном порядке, может извлекать из общего входного потока в нужном алгоритму порядке. Хорошим тоном является обозначение идентификаторов символьными именами посредством операторов "#define" или "const int".
- Описатель области связи (коммуникатор). Обязан быть одинаковым для MPI_Send и MPI_Recv.
- Статус завершения приема. Содержит информацию о принятом сообщении: его идентификатор, номер задачи-передатчика, код завершения и количество фактически пришедших данных.

Справочная информация по MPI

- Страница MPI на сервере ANL. Авторы: Bill Gropp, Rusty Lusk <http://www.mcs.anl.gov/mpi/>
- PMB (Pallas MPI Benchmarks). Набор тестов оценки производительности реализаций MPI.
<http://www.pallas.de/pages/pmb.htm>
- Tuning MPI Applications for Peak Performance.
Руководство по оптимизации MPI-приложений в формате Powerpoint. Автор: Bill Gropp.
<http://www.mcs.anl.gov/Projects/mpi/tutorials/perf/>

Пошаговое руководство. Запуск отладчика MPI-кластера в MS Visual Studio

- https://msdn.microsoft.com/ru-ru/library/ee441265.aspx#BKMK_reqs